

Prototypes des interfaces et contrôleurs - Système de Ticketing

1. Votre partie : Gestion des Tickets

Interfaces de services

ITicketService

csharp

```
public interface ITicketService
{
    Task<Ticket> CreateTicketAsync(Ticket ticket);
    Task<Ticket> GetTicketByIdAsync(int id);
    Task<IEnumerable<Ticket>> GetTicketsByUserIdAsync(string userId);
    Task<IEnumerable<Ticket>> GetTicketsByTeamIdAsync(int teamId);
    Task<IEnumerable<Ticket>> GetTicketsByStatusAsync(TicketStatus status);
    Task<IEnumerable<Ticket>> GetTicketsByCategoryAsync(TicketCategory category);
    Task<IEnumerable<Ticket>> GetTicketsByCreatorIdAsync(string creatorId);
    Task<IEnumerable<Ticket>> GetAllTicketsAsync();
    Task<Ticket> UpdateTicketAsync(Ticket ticket);
    Task<bool> DeleteTicketAsync(int id);
    Task<bool> AddCommentAsync(TicketComment comment);
    Task<bool> AddAttachmentAsync(Attachment attachment);
}
```

ITicketAssignmentService

csharp

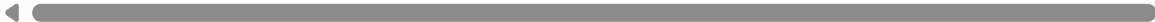
```
public interface ITicketAssignmentService
{
    Task AssignTicketAutomatically(Ticket ticket);
    Task<bool> AssignTicketToUserAsync(int ticketId, string userId, string assignedByUserId);
    Task<bool> AssignTicketToTeamAsync(int ticketId, int teamId, string assignedByUserId);
    Task<bool> UnassignTicketAsync(int ticketId, string unassignedByUserId);
    Task<IEnumerable<AssignmentRule>> GetAssignmentRulesAsync();
    Task<AssignmentRule> GetAssignmentRuleByIdAsync(int ruleId);
    Task<bool> AddAssignmentRuleAsync(AssignmentRule rule);
    Task<bool> UpdateAssignmentRuleAsync(AssignmentRule rule);
    Task<bool> DeleteAssignmentRuleAsync(int ruleId);
    Task<bool> UpdateRulePriorityAsync(int ruleId, int newPriority);
}
```



IEmailTicketService

csharp

```
public interface IEmailTicketService
{
    Task ProcessIncomingEmailAsync(string subject, string body, string senderEmail, List<string> attachments);
    Task SendTicketUpdateEmailAsync(int ticketId, TicketStatus newStatus, string comment = null);
    Task SendTicketAssignmentEmailAsync(int ticketId, string assignedToUserId);
    Task SendTicketCommentEmailAsync(int ticketId, int commentId);
    Task SendTicketEscalationEmailAsync(int ticketId, string escalatedToUserId);
    Task<bool> ConfigureEmailSettingsAsync(EmailSettings settings);
    Task<EmailSettings> GetEmailSettingsAsync();
}
```



ITicketHistoryService

csharp

```
public interface ITicketHistoryService
{
    Task AddHistoryEntryAsync(TicketHistory historyEntry);
    Task<IEnumerable<TicketHistory>> GetTicketHistoryAsync(int ticketId);
    Task<IEnumerable<TicketHistory>> GetTicketHistoryByFieldAsync(int ticketId, string fieldName);
    Task<IEnumerable<TicketHistory>> GetTicketHistoryByUserAsync(int ticketId, string userId);
    Task<IEnumerable<TicketHistory>> GetTicketHistoryByDateRangeAsync(int ticketId, DateTime startDate, DateTime endDate);
}
```



Prototypes des contrôleurs

TicketsController

csharp

```
[Authorize]
public class TicketsController : Controller
{
    // GET: Tickets
    public Task<IActionResult> Index();

    // GET: Tickets/Details/5
    public Task<IActionResult> Details(int id);

    // GET: Tickets/Create
    public IActionResult Create();

    // POST: Tickets/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Create(TicketCreateViewModel model);

    // GET: Tickets/Edit/5
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> Edit(int id);

    // POST: Tickets/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> Edit(int id, TicketEditViewModel model);

    // POST: Tickets/AddComment
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> AddComment(AddCommentViewModel model);

    // POST: Tickets/ChangeStatus
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> ChangeStatus(int id, TicketStatus newStatus);

    // POST: Tickets/AssignToUser
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> AssignToUser(int id, string userId);
```

```

// POST: Tickets/AssignToTeam
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin,SupportAgent")]
public Task<IActionResult> AssignToTeam(int id, int teamId);

// GET: Tickets/DownloadAttachment/5
public Task<IActionResult> DownloadAttachment(int attachmentId);

// POST: Tickets/AddAttachment
[HttpPost]
[ValidateAntiForgeryToken]
public Task<IActionResult> AddAttachment(int ticketId, IFormFile file);

// GET: Tickets/MyTickets
public Task<IActionResult> MyTickets();

// GET: Tickets/TeamTickets
[Authorize(Roles = "Admin,SupportAgent")]
public Task<IActionResult> TeamTickets(int teamId);

// GET: Tickets/Search
public Task<IActionResult> Search(TicketSearchViewModel model);
}

```

AssignmentRulesController

csharp

```
[Authorize(Roles = "Admin")]
public class AssignmentRulesController : Controller
{
    // GET: AssignmentRules
    public Task<IActionResult> Index();

    // GET: AssignmentRules/Details/5
    public Task<IActionResult> Details(int id);

    // GET: AssignmentRules/Create
    public IActionResult Create();

    // POST: AssignmentRules/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Create(AssignmentRuleViewModel model);

    // GET: AssignmentRules/Edit/5
    public Task<IActionResult> Edit(int id);

    // POST: AssignmentRules/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Edit(int id, AssignmentRuleViewModel model);

    // POST: AssignmentRules/Delete/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Delete(int id);

    // POST: AssignmentRules/UpdatePriority
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> UpdatePriority(int id, int newPriority);

    // POST: AssignmentRules/ToggleActive
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> ToggleActive(int id);
}
```

EmailSettingsController

csharp

```
[Authorize(Roles = "Admin")]
public class EmailSettingsController : Controller
{
    // GET: EmailSettings
    public Task<IActionResult> Index();

    // POST: EmailSettings/Configure
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Configure(EmailSettingsViewModel model);

    // GET: EmailSettings/TestConnection
    public Task<IActionResult> TestConnection();

    // GET: EmailSettings/EmailTemplates
    public Task<IActionResult> EmailTemplates();

    // POST: EmailSettings/UpdateTemplate
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> UpdateTemplate(EmailTemplateViewModel model);
}
```

2. Partie de Walid : Notifications et Gestion des Équipes

Interfaces de services

INotificationService

csharp

```
public interface INotificationService
{
    Task<Notification> CreateNotificationAsync(string userId, string title, string message);
    Task<IEnumerable<Notification>> GetUserNotificationsAsync(string userId, bool unreadOnly =
    Task<bool> MarkNotificationAsReadAsync(int notificationId);
    Task<bool> MarkAllNotificationsAsReadAsync(string userId);
    Task<bool> DeleteNotificationAsync(int notificationId);
    Task<bool> DeleteAllNotificationsAsync(string userId);
    Task<int> GetUnreadNotificationCountAsync(string userId);
    Task SendTicketAssignmentNotification(string userId, int ticketId, bool isTeamAssignment, i
    Task SendTicketStatusNotification(string userId, int ticketId, TicketStatus newStatus);
    Task SendTicketCommentNotification(string userId, int ticketId, string message);
    Task SendEscalationNotification(IEnumerable<string> userIds, int ticketId);
    Task<bool> ConfigureUserNotificationPreferences(string userId, NotificationPreferences pref
    Task<NotificationPreferences> GetUserNotificationPreferences(string userId);
}
```

ITeamService

csharp

```
public interface ITeamService
{
    Task<SupportTeam> CreateTeamAsync(SupportTeam team);
    Task<SupportTeam> GetTeamByIdAsync(int id);
    Task<IEnumerable<SupportTeam>> GetAllTeamsAsync();
    Task<SupportTeam> UpdateTeamAsync(SupportTeam team);
    Task<bool> DeleteTeamAsync(int id);
    Task<bool> AddMemberToTeamAsync(int teamId, string userId);
    Task<bool> RemoveMemberFromTeamAsync(int teamMemberId);
    Task<bool> UpdateTeamManagerAsync(int teamId, string managerId);
    Task<IEnumerable<User>> GetTeamMembersAsync(int teamId);
    Task<IEnumerable<Ticket>> GetTeamTicketsAsync(int teamId);
    Task<IEnumerable<SupportTeam>> GetUserTeamsAsync(string userId);
    Task<IEnumerable<TeamPerformanceMetrics>> GetTeamPerformanceMetricsAsync(int teamId, DateTi
    Task<IEnumerable<User>> GetAvailableAgentsForTeamAsync(int teamId);
    Task<IEnumerable<TeamMember>> GetTeamMembershipHistoryAsync(string userId);
}
```

IEscalationService

csharp

```
public interface IEscalationService
{
    Task CheckAndEscalateTicketsAsync();
    Task<bool> CreateEscalationRuleAsync(EscalationRule rule);
    Task<bool> UpdateEscalationRuleAsync(EscalationRule rule);
    Task<bool> DeleteEscalationRuleAsync(int ruleId);
    Task<IEnumerable<EscalationRule>> GetAllEscalationRulesAsync();
    Task<EscalationRule> GetEscalationRuleByIdAsync(int ruleId);
    Task<bool> ToggleEscalationRuleStatusAsync(int ruleId);
    Task<bool> ManuallyEscalateTicketAsync(int ticketId, string userId, int? teamId);
    Task<IEnumerable<Ticket>> GetEscalatedTicketsAsync();
    Task<IEnumerable<Ticket>> GetTicketsApproachingEscalationAsync();
    Task<EscalationHistory> GetTicketEscalationHistoryAsync(int ticketId);
    Task<bool> UpdateEscalationRulePriorityAsync(int ruleId, int newPriority);
}
```

Prototypes des contrôleurs

NotificationsController

csharp

```
[Authorize]
public class NotificationsController : Controller
{
    // GET: Notifications
    public Task<IActionResult> Index();

    // GET: Notifications/GetUnreadCount
    public Task<JsonResult> GetUnreadCount();

    // GET: Notifications/GetLatestNotifications
    public Task<JsonResult> GetLatestNotifications(int count = 5);

    // POST: Notifications/MarkAsRead
    [HttpPost]
    public Task<IActionResult> MarkAsRead(int id);

    // POST: Notifications/MarkAllAsRead
    [HttpPost]
    public Task<IActionResult> MarkAllAsRead();

    // POST: Notifications/Delete
    [HttpPost]
    public Task<IActionResult> Delete(int id);

    // POST: Notifications/DeleteAll
    [HttpPost]
    public Task<IActionResult> DeleteAll();

    // GET: Notifications/Preferences
    public Task<IActionResult> Preferences();

    // POST: Notifications/UpdatePreferences
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> UpdatePreferences(NotificationPreferencesViewModel model);
}
```

TeamsController

csharp

```
[Authorize(Roles = "Admin")]
public class TeamsController : Controller
{
    // GET: Teams
    public Task<IActionResult> Index();

    // GET: Teams/Details/5
    public Task<IActionResult> Details(int id);

    // GET: Teams/Create
    public IActionResult Create();

    // POST: Teams/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Create(TeamViewModel model);

    // GET: Teams/Edit/5
    public Task<IActionResult> Edit(int id);

    // POST: Teams/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Edit(int id, TeamViewModel model);

    // POST: Teams/Delete/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Delete(int id);

    // GET: Teams/AddMember/5
    public Task<IActionResult> AddMember(int id);

    // POST: Teams/AddMember
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> AddMember(int teamId, string userId);

    // POST: Teams/RemoveMember
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> RemoveMember(int teamMemberId);
```

```
// GET: Teams/TeamTickets/5
public Task<IActionResult> TeamTickets(int id);

// GET: Teams/Performance/5
public Task<IActionResult> Performance(int id, DateTime? startDate, DateTime? endDate);

// GET: Teams/MemberPerformance/5
public Task<IActionResult> MemberPerformance(int teamId, string userId);

// GET: Teams/TeamManagement
[Authorize(Roles = "Admin,SupportAgent")]
public Task<IActionResult> TeamManagement();
}
```

EscalationRulesController

csharp

```
[Authorize(Roles = "Admin")]
public class EscalationRulesController : Controller
{
    // GET: EscalationRules
    public Task<IActionResult> Index();

    // GET: EscalationRules/Details/5
    public Task<IActionResult> Details(int id);

    // GET: EscalationRules/Create
    public IActionResult Create();

    // POST: EscalationRules/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Create(EscalationRuleViewModel model);

    // GET: EscalationRules/Edit/5
    public Task<IActionResult> Edit(int id);

    // POST: EscalationRules/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Edit(int id, EscalationRuleViewModel model);

    // POST: EscalationRules/Delete/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> Delete(int id);

    // POST: EscalationRules/ToggleActive/5
    [HttpPost]
    public Task<IActionResult> ToggleActive(int id);

    // GET: EscalationRules/EscalatedTickets
    public Task<IActionResult> EscalatedTickets();

    // GET: EscalationRules/TicketsApproachingEscalation
    public Task<IActionResult> TicketsApproachingEscalation();

    // POST: EscalationRules/ManualEscalation
    [HttpPost]
    [ValidateAntiForgeryToken]
```



```

[Authorize(Roles = "Admin,SupportAgent")]
public Task<IActionResult> ManualEscalation(ManualEscalationViewModel model);

// GET: EscalationRules/EscalationHistory/5
public Task<IActionResult> EscalationHistory(int ticketId);
}

```

3. Partie de Youssef : Tableaux de bord et Reporting

Interfaces de services

IDashboardService

```

csharp

public interface IDashboardService
{
    Task<AdminDashboardViewModel> GetAdminDashboardDataAsync();
    Task<AgentDashboardViewModel> GetAgentDashboardDataAsync(string userId);
    Task<UserDashboardViewModel> GetUserDashboardDataAsync(string userId);
    Task<TeamPerformanceViewModel> GetTeamPerformanceDataAsync(int teamId);
    Task<TicketMetricsViewModel> GetTicketMetricsAsync(DateTime? startDate = null, DateTime? endDate = null);
    Task<IDictionary<string, int>> GetTicketsByStatusAsync();
    Task<IDictionary<string, int>> GetTicketsByCategoryAsync();
    Task<IDictionary<string, int>> GetTicketsByPriorityAsync();
    Task<IEnumerable<DailyTicketCount>> GetDailyTicketCountsAsync(int days = 30);
    Task<IEnumerable<AgentPerformanceMetrics>> GetAgentPerformanceMetricsAsync();
    Task<IEnumerable<TeamPerformanceMetrics>> GetTeamPerformanceMetricsAsync();
    Task<IEnumerable<Ticket>> GetRecentTicketsAsync(int count = 10);
    Task<IEnumerable<TicketResponseTime>> GetAverageResponseTimesAsync();
    Task<IEnumerable<Ticket>> GetSlaBreachedTicketsAsync();
}

```

IReportService

csharp

```
public interface IReportService
{
    Task<byte[]> GenerateTicketsReportPdfAsync(TicketReportParameters parameters);
    Task<byte[]> GenerateTicketsReportExcelAsync(TicketReportParameters parameters);
    Task<byte[]> GenerateTeamPerformanceReportAsync(int teamId, DateTime startDate, DateTime endDate);
    Task<byte[]> GenerateUserPerformanceReportAsync(string userId, DateTime startDate, DateTime endDate);
    Task<byte[]> GenerateSlaComplianceReportAsync(DateTime startDate, DateTime endDate);
    Task<byte[]> GenerateCustomReportAsync(CustomReportParameters parameters);
    Task<IEnumerable<ReportTemplate>> GetAvailableReportTemplatesAsync();
    Task<ReportTemplate> GetReportTemplateByIdAsync(int templateId);
    Task<bool> SaveReportTemplateAsync(ReportTemplate template);
    Task<bool> DeleteReportTemplateAsync(int templateId);
    Task<string> ScheduleRecurringReportAsync(RecurringReportParameters parameters);
    Task<IEnumerable<ScheduledReport>> GetScheduledReportsAsync();
    Task<bool> CancelScheduledReportAsync(string scheduleId);
}
```

ISlaService

csharp

```
public interface ISlaService
{
    Task<IEnumerable<Ticket>> GetTicketsAtRiskOfSlaBreachAsync();
    Task<IEnumerable<Ticket>> GetSlaBreachedTicketsAsync();
    Task<SlaPerformanceViewModel> GetSlaPerformanceDataAsync(DateTime startDate, DateTime endDate);
    Task<bool> IsTicketWithinSlaAsync(int ticketId);
    Task<TimeSpan> GetRemainingTimeForTicketSlaAsync(int ticketId);
    Task<IEnumerable<SlaDefinition>> GetSlaDefinitionsAsync();
    Task<SlaDefinition> GetSlaDefinitionByIdAsync(int slaId);
    Task<bool> CreateSlaDefinitionAsync(SlaDefinition sla);
    Task<bool> UpdateSlaDefinitionAsync(SlaDefinition sla);
    Task<bool> DeleteSlaDefinitionAsync(int slaId);
    Task<SlaStats> GetSlaStatsForTicketAsync(int ticketId);
    Task<SlaStats> GetSlaStatsForTeamAsync(int teamId);
    Task<SlaStats> GetSlaStatsForUserAsync(string userId);
    Task<IEnumerable<SlaBreachReport>> GetSlaBreachReportsAsync(DateTime startDate, DateTime endDate);
}
```

Prototypes des contrôleurs

csharp

```
[Authorize]
public class DashboardController : Controller
{
    // GET: Dashboard
    public Task<IActionResult> Index();

    // GET: Dashboard/Admin
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> Admin();

    // GET: Dashboard/Agent
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> Agent();

    // GET: Dashboard/User
    public Task<IActionResult> User();

    // GET: Dashboard/TeamPerformance/5
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<IActionResult> TeamPerformance(int teamId);

    // GET: Dashboard/TicketMetrics
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> TicketMetrics(DateTime? startDate, DateTime? endDate);

    // API Methods for AJAX calls

    // GET: Dashboard/GetTicketsByStatus
    public Task<JsonResult> GetTicketsByStatus();

    // GET: Dashboard/GetTicketsByCategory
    public Task<JsonResult> GetTicketsByCategory();

    // GET: Dashboard/GetTicketsByPriority
    public Task<JsonResult> GetTicketsByPriority();

    // GET: Dashboard/GetDailyTicketCounts
    public Task<JsonResult> GetDailyTicketCounts(int days = 30);

    // GET: Dashboard/GetAgentPerformance
    [Authorize(Roles = "Admin,SupportAgent")]
    public Task<JsonResult> GetAgentPerformance();
```

```
// GET: Dashboard/GetTeamPerformance
[Authorize(Roles = "Admin,SupportAgent")]
public Task<JsonResult> GetTeamPerformance();

// GET: Dashboard/GetRecentTickets
public Task<JsonResult> GetRecentTickets(int count = 10);

// GET: Dashboard/GetAverageResponseTimes
[Authorize(Roles = "Admin,SupportAgent")]
public Task<JsonResult> GetAverageResponseTimes();

// GET: Dashboard/GetSlaBreachedTickets
[Authorize(Roles = "Admin,SupportAgent")]
public Task<JsonResult> GetSlaBreachedTickets();
}
```

ReportsController

csharp

```

[Authorize(Roles = "Admin,SupportAgent")]
public class ReportsController : Controller
{
    // GET: Reports
    public Task<IActionResult> Index();

    // GET: Reports/TicketsReport
    public Task<IActionResult> TicketsReport();

    // POST: Reports/GenerateTicketsReport
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> GenerateTicketsReport(TicketsReportViewModel model);

    // GET: Reports/TeamPerformance
    public Task<IActionResult> TeamPerformance();

    // POST: Reports/GenerateTeamPerformanceReport
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> GenerateTeamPerformanceReport(TeamPerformanceReportViewModel model);

    // GET: Reports/AgentPerformance
    public Task<IActionResult> AgentPerformance();

    // POST: Reports/GenerateAgentPerformanceReport
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> GenerateAgentPerformanceReport(AgentPerformanceReportViewModel model);

    // GET: Reports/SLaCompliance
    public Task<IActionResult> SLaCompliance();

    // POST: Reports/GenerateSLaComplianceReport
    [HttpPost]
    [ValidateAntiForgeryToken]
    public Task<IActionResult> GenerateSLaComplianceReport(SLaReportViewModel model);

    // GET: Reports/CustomReport
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> CustomReport();

    // POST: Reports/GenerateCustomReport

```



```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
public Task<IActionResult> GenerateCustomReport(CustomReportViewModel model);

// GET: Reports/ReportTemplates
[Authorize(Roles = "Admin")]
public Task<IActionResult> ReportTemplates();

// GET: Reports/CreateTemplate
[Authorize(Roles = "Admin")]
public Task<IActionResult> CreateTemplate();

// POST: Reports/SaveTemplate
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
public Task<IActionResult> SaveTemplate(ReportTemplateViewModel model);

// POST: Reports/DeleteTemplate
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
public Task<IActionResult> DeleteTemplate(int templateId);

// GET: Reports/ScheduledReports
[Authorize(Roles = "Admin")]
public Task<IActionResult> ScheduledReports();

// GET: Reports/ScheduleReport
[Authorize(Roles = "Admin")]
public Task<IActionResult> ScheduleReport();

// POST: Reports/CreateScheduledReport
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
public Task<IActionResult> CreateScheduledReport(ScheduledReportViewModel model);

// POST: Reports/CancelScheduledReport
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin")]
public Task<IActionResult> CancelScheduledReport(string scheduleId);
```

```
// GET: Reports/DownloadPdf
public Task<FileResult> DownloadPdf(string reportId);

// GET: Reports/DownloadExcel
public Task<FileResult> DownloadExcel(string reportId);
}
```

SlaController

csharp

```
[Authorize(Roles = "Admin,SupportAgent")]
public class SlaController : Controller
{
    // GET: Sla
    public Task<IActionResult> Index();

    // GET: Sla/Definitions
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> Definitions();

    // GET: Sla/CreateDefinition
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> CreateDefinition();

    // POST: Sla/CreateDefinition
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> CreateDefinition(SlaDefinitionViewModel model);

    // GET: Sla/EditDefinition/5
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> EditDefinition(int id);

    // POST: Sla/EditDefinition/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> EditDefinition(int id, SlaDefinitionViewModel model);

    // POST: Sla/DeleteDefinition/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin")]
    public Task<IActionResult> DeleteDefinition(int id);

    // GET: Sla/TicketsAtRisk
    public Task<IActionResult> TicketsAtRisk();

    // GET: Sla/BreachedTickets
    public Task<IActionResult> BreachedTickets();

    // GET: Sla/Performance
```

```
public Task<IActionResult> Performance(DateTime? startDate, DateTime? endDate);

// GET: Sla/TicketStats/5
public Task<IActionResult> TicketStats(int ticketId);

// GET: Sla/TeamStats/5
public Task<IActionResult> TeamStats(int teamId);

// GET: Sla/UserStats/5
public Task<IActionResult> UserStats(string userId);

// GET: Sla/BreachReports
[Authorize(Roles = "Admin")]
public Task<IActionResult> BreachReports(DateTime? startDate, DateTime? endDate);

// API Methods for AJAX calls

// GET: Sla/GetSlaStatusForTicket/5
public Task<JsonResult> GetSlaStatusForTicket(int ticketId);

// GET: Sla/GetRemainingTimeForTicket/5
public Task<JsonResult> GetRemainingTimeForTicket(int ticketId);

// GET: Sla/GetSlaPerformanceData
public Task<JsonResult> GetSlaPerformanceData(
```